Tydenso

Symbolic tensor algebra inside Typst

User manual · version 0.1.0

Build indexed tensors as readable Typst values, transform them with Idenso, and print them with Spenso's own notation.

Repository · MIT license · Spenso Python API · Idenso Python API

Contents

1 Start with a contraction

Tydenso is the tensor-focused companion to Tymbolica. It has its own package, manual, printer, and WebAssembly engine. You do not need Tymbolica to construct, simplify, inspect, or display a tensor expression.

The first example contracts a Minkowski metric with a vector. A representation is an ordinary Typst dictionary; `slot` turns an index label into another dictionary; `vector` creates a callable rank-one tensor name.

```
#let V = mink(4)
#let mu = slot(V, "mu")
#let nu = slot(V, "nu")
#let p = vector("p")

#let before = mul(metric(V, mu, nu), p(nu))
#let after = simplify-metrics(before)

$
  #to-typst(before)
  quad arrow.r.long
  #to-typst(after)
$
```

$$p^{\nu} g^{\alpha_{\mathrm{mu}}\alpha_{\mathrm{nu}}} \quad \longrightarrow p^{\mu}$$

The expression itself is a lossless Symbolica Atom payload. Keep that byte value for algebra; call `to-typst` where the result belongs on the page.

1.1 Installation

Until Tydenso is published separately, install the `tydenso` directory from this repository as its own local package. On Linux, place or symlink it at:

`~/.local/share/typst/packages/local/tydenso/0.1.0`

Then import it independently:

```
#import "@local/tydenso:0.1.0": *
```

From a checkout, `#import "path/to/tymbolica/tydenso/lib.typ": *` works as well. The package directory already contains its compressed engine and loader.

1.2 How this corresponds to the Python API

The public concepts follow Spenso's Python surface, adapted to what is natural in Typst. Python objects can overload calls; Typst dictionaries cannot, so index construction is the explicit `slot(V, "mu")`. Tensor names are functions, which keeps the pleasant `p(mu)` spelling.

Python	Typst

`V = Representation.mink(4)`
`mu = V("mu")`
`nu = V("nu")`
`p = TensorName("p")`
`expr = TensorName.g(mu, nu) * p(nu)`
	`#let V = mink(4)`
	`#let mu = slot(V, "mu")`
	`#let nu = slot(V, "nu")`
	`#let p = vector("p")`
	`#let expr = mul(metric(V, mu, nu), p(nu))`

The Typst representation and slot values stay transparent; only a completed symbolic expression becomes Atom bytes.

2 Build tensor expressions

2.1 Representations and slots are data

Built-in constructors cover the representations initialized by Spenso and Idenso: `mink`, `euc`, `lor`, `bis`, `spf`, `cof`, `coad`, and `cos`. Use `representation` when a model introduces another representation.

```
#let color = cof(3)
#let a = slot(color, "a")
#let b-lower = slot(color, "b", dual: true)
```

```
#table(
  columns: (auto, 1fr),
  inset: 5pt,
  [representation], [#color.name],
  [dimension], [#color.dimension],
  [first index], [#a.index],
  [second index is dual], [#b-lower.dual],
)
```

representation	cof
dimension	3
first index	a
second index is dual	true

Because those values are dictionaries, document code can validate dimensions, generate index families in a loop, or attach its own metadata before asking the plugin to construct an Atom. `dual-representation` returns the paired representation; `metric`, `identity-tensor`, and `flat-tensor` accept either index labels or already constructed slots.

2.2 Tensor names carry symmetry

`tensor` mirrors the useful part of Spenso's `TensorName`: a name plus optional symmetric, antisymmetric, cyclic, or linear behavior. Symbolica applies those attributes while it constructs the function, not as a later cosmetic step.

```
#let V = euc(3)
#let mu = slot(V, "mu")
#let nu = slot(V, "nu")
#let F = tensor("F", antisymmetric: true)

#let cancellation = add(F(mu, nu), F(nu, mu))
$ F_(mu nu) + F_(nu mu) = #to-typst(cancellation) $
```

$$F\left(\begin{array}{l}\text{kind: "slot",}\\ \text{representation: (}\\ \text{kind: "representation",}\\ \text{name: "euc",}\\ \text{namespace: "spenso",}\\ \text{dimension: 3,}\\ \text{self-dual: true,}\\ \text{is-dual: false,}\\ \text{dual-name: "euc",}\\ \text{),}\\ \text{index: "mu",}\\ \text{dual: false,}\end{array}\right)\left(\begin{array}{l}\text{kind: "slot",}\\ \text{representation: (}\\ \text{kind: "representation",}\\ \text{name: "euc",}\\ \text{namespace: "spenso",}\\ \text{dimension: 3,}\\ \text{self-dual: true,}\\ \text{is-dual: false,}\\ \text{dual-name: "euc",}\\ \text{),}\\ \text{index: "nu",}\\ \text{dual: false,}\end{array}\right) + F\left(\begin{array}{l}\text{kind: "slot",}\\ \text{representation: (}\\ \text{kind: "representation",}\\ \text{name: "euc",}\\ \text{namespace: "spenso",}\\ \text{dimension: 3,}\\ \text{self-dual: true,}\\ \text{is-dual: false,}\\ \text{dual-name: "euc",}\\ \text{),}\\ \text{index: "nu",}\\ \text{dual: false,}\end{array}\right)\left(\begin{array}{l}\text{kind: "slot",}\\ \text{representation: (}\\ \text{kind: "representation",}\\ \text{name: "euc",}\\ \text{namespace: "spenso",}\\ \text{dimension: 3,}\\ \text{self-dual: true,}\\ \text{is-dual: false,}\\ \text{dual-name: "euc",}\\ \text{),}\\ \text{index: "mu",}\\ \text{dual: false,}\end{array}\right) = 0$$

Only one of `symmetric`, `antisymmetric`, and `cycle-symmetric` may be true for one tensor name. The `linear` flag is independent.

2.3 Compose without parsing strings

`add`, `mul`, `neg`, `sub`, `div`, and `pow` accept Atom payloads together with integers, floats, strings, slots, and representation dictionaries. Strings are symbol names in the `spenso` namespace; use `symbol` to choose another namespace explicitly.

```
#let V = mink("D")
#let mu = slot(V, "mu")
#let p = vector("p")
#let mass = symbol("m", namespace: "model")

#let shell = sub(pow(p(mu), 2), pow(mass, 2))
$ #to-typst(shell) $
```

$$-m^2 + p^{\mu\,2}$$

This constructor path is intentionally structural. It does not depend on a Typst-math parser guessing which arguments are tensor slots.

3 Transform tensors

Idenso provides the domain-specific transformations. The most common ones simplify metrics, gamma matrices, and color structures; selective expanders and index-wrapping functions are available for lower-level workflows.

3.1 Contract a metric chain

The next expression contains two metrics and one vector. Simplifying metrics eliminates both contracted dummy indices in one pass.

```
#let V = mink(4)
#let mu = slot(V, "mu")
#let nu = slot(V, "nu")
#let rho = slot(V, "rho")
#let p = vector("p")

#let chain = mul(
  metric(V, mu, nu),
  metric(V, nu, rho),
  p(rho),
)
#let reduced = simplify-metrics(chain)

$
  #to-typst(chain)
  quad arrow.r.long
  #to-typst(reduced)
$
```

$$p^{\rho}\, g^{\alpha_{\mathrm{mu}}\alpha_{\mathrm{nu}}}\, g^{\alpha_{\mathrm{nu}}\alpha_{\mathrm{rho}}} \quad \longrightarrow \quad p^{\mu}$$

`list-dangling` returns the free indices as Atom payloads. Pass an element to `to-typst`, `to-string`, or `inspect` just like any other expression.

3.2 Know which family to call

Family	Purpose
`simplify-metrics`, `expand-metrics`, `expand-mink`, `expand-bis`	Metric, Minkowski, and bispinor structure.
`simplify-gamma`, `dirac-adjoint`	Dirac chains and conjugation.
`simplify-color`, `expand-color`	Color generators and structure constants.
`cook-function`, `cook-indices`, `wrap-dummies`, `wrap-indices`	Canonical index organization and explicit wrappers.
`to-dots`	Rewrite supported contractions as dot products.

The mathematical conventions and supported identities track Idenso's API.

4 Print and inspect

4.1 Use the Spenso printer directly

Tydenso does not send tensor output through Tymbolica's general printer. `to-typst-source` uses Symbolica's Typst arithmetic mode together with Spenso's custom tensor notation. `to-string` uses Spenso's compact Symbolica notation.

```
#let V = mink(4)
#let p = vector("p")
#let expression = p(1, slot(V, "mu"))

#let detailed = print-settings(with-dim: true, commas: true)

Typst source:
#raw(to-typst-source(expression, settings: detailed), block: true)

Compact Spenso form:
#raw(to-string(expression), block: true)

Rendered: $ #to-typst(expression, settings: detailed) $
```

```
Typst source:
attach(#($p$,std.hide($zws$)).join(),t:std.hide(1) mu,b:1 std.hide(mu))
Compact Spenso form:
p(1,αmu)
Rendered:
```

$$p_1{}^{\mu}$$

In Typst mode, tensor and vector heads use a single `attach` expression. Upper and lower scripts occupy matching columns with hidden counterparts, following Physica's alignment technique. A plain self-dual slot is placed on the top row; self-dual variance wrappers normalize away because the representation is its own dual. For a dualizable representation, its dual orientation is placed on the bottom row.

`print-settings` exposes the real `SpensoPrintSettings` switches: `with-dim`, `parens`, `commas`, `index-subscripts`, and `symbol-scripts`. Start from either the `"typst"` or `"compact"` preset and override only what the document needs.

4.2 Inspect the Atom as CBOR data

Atom bytes remain the lossless interchange format, but they are not the only view. `inspect` decodes an expression to recursive Typst data. Every node has a `kind`; function nodes also expose their full and short names, arguments, and symmetry flags.

```
#let V = euc(3)
#let A = tensor("A", symmetric: true)
#let expression = A(slot(V, "i"), slot(V, "j"))
#let tree = inspect(expression)

#table(
  columns: (auto, 1fr),
  inset: 5pt,
  [node kind], [#tree.kind],
  [function], [#tree.short-name],
  [arguments], [#tree.arguments.len()],
  [symmetric], [#tree.symmetric],
)
```

node kind	function
function	A
arguments	2
symmetric	true

Why keep both forms? CBOR is convenient for layouts, diagnostics, and package-level APIs. Reconstructing algebra from that tree would lose Symbolica state and exact coefficient details, so transformations continue to consume Atom bytes.

5 Work alongside Tymbolica

Tydenso and Tymbolica use the same versioned native Atom export. A package can construct and simplify tensors with Tydenso, then pass the result to Tymbolica for a general algebraic operation. It can also go the other way when a Tymbolica expression uses Spenso's tensor representation.

```
#import "@local/tydenso:0.1.0" as tensors
#import "@local/tymbolica:0.1.0" as algebra

#let V = tensors.mink(4)
#let p = tensors.vector("p")
#let expression = p(tensors.slot(V, "mu"))

#let expanded = algebra.expand(expression)
#tensors.to-typst(expanded)
```

Keep package versions aligned. Matrix payloads from Tymbolica are a different format and are not Tydenso inputs.

6 API reference

The groups below cover the imported top-level API. `init` returns the same surface as a dictionary bound to a selected plugin module.

6.1 Tensor construction

6.1.1 tensor

Construct a named tensor function.

The result is callable with any mixture of slots, representations, scalar values, and compatible Atom payloads. Symmetry flags follow Spenso's `TensorName` model and are registered on the underlying Symbolica symbol.

```
#let V = mink(4)
#let Z = tensor("Z")
#to-typst(Z(slot(V, "mu"), slot(V, "nu")))
```

$$Z^{\mu\nu}$$

Parameters

```
tensor(
  name: str,
  namespace: str,
  symmetric: bool,
  antisymmetric: bool,
  cycle-symmetric: bool,
  linear: bool
) -> function
```

6.1.1.0.1 name `str`

Tensor name.

6.1.1.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

6.1.1.0.3 symmetric `bool`

Make the tensor symmetric under argument permutation.

Default: `false`

6.1.1.0.4 antisymmetric `bool`

Make the tensor antisymmetric under argument permutation.

Default: `false`

6.1.1.0.5 cycle-symmetric `bool`

Make the tensor symmetric under cyclic argument permutation.

Default: `false`

6.1.1.0.6 linear `bool`

Mark the tensor function as linear.

Default: `false`

6.1.2 vector

Construct a named rank-one tensor function.

Non-slot arguments are rendered as symbol subscripts in Spenso's Typst mode, while the vector slot is rendered as an abstract index.

```
#let V = mink(4)
#let p = vector("p")
#to-typst(p(1, slot(V, "mu")))
```

$$p_1{}^{\mu}$$

Parameters

```
vector(
  name: str,
  namespace: str,
  linear: bool
) -> function
```

6.1.2.0.1 name `str`

Vector name.

6.1.2.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

6.1.2.0.3 linear `bool`

Mark the vector function as linear.

Default: `false`

6.1.3 symbol

Construct a symbolic scalar.

Parameters

```
symbol(
  name: str,
  namespace: str
) -> bytes
```

6.1.3.0.1 name `str`

Symbol name.

6.1.3.0.2 namespace `str`

Symbol namespace.

Default: `"spenso"`

6.1.4 representation

Describe a representation and attach its tensor-building methods.

The returned dictionary is inspectable Typst data. Its `slot`, `metric`, `identity`, `flat`, and `dual` fields are functions.

Parameters

```
representation(
  name: str,
  dimension: int str bytes,
  namespace: str,
  self-dual: bool,
  is-dual: bool,
  dual-name: str none
) -> dictionary
```

6.1.4.0.1 name `str`

Symbolic representation name.

6.1.4.0.2 dimension `int` or `str` or `bytes`

Dimension, which may itself be symbolic.

6.1.4.0.3 namespace `str`

Symbol namespace.

Default: `"spenso"`

6.1.4.0.4 self-dual `bool`

Whether the representation is its own dual.

Default: `false`

6.1.4.0.5 is-dual `bool`

Whether this value denotes the dual orientation of a non-self-dual representation.

Default: `false`

6.1.4.0.6 dual-name `str` or `none`

Name used by the representation returned from `dual`.

Default: `none`

6.1.5 mink

Construct a Minkowski representation.

Parameters

```
mink(dimension) -> dictionary
```

6.1.6 euc

Construct a Euclidean representation.

Parameters

```
euc(dimension) -> dictionary
```

6.1.7 lor

Construct a Lorentz representation.

Parameters

```
lor(dimension) -> dictionary
```

6.1.8 bis

Construct a bispinor representation.

Parameters

```
bis(dimension) -> dictionary
```

6.1.9 spf

Construct a spin-fundamental representation.

Parameters

```
spf(dimension) -> dictionary
```

6.1.10 cof

Construct a color-fundamental representation.

Parameters

```
cof(dimension) -> dictionary
```

6.1.11 coad

Construct a color-adjoint representation.

Parameters

```
coad(dimension) -> dictionary
```

6.1.12 cos

Construct a color-sextet representation.

Parameters

```
cos(dimension) -> dictionary
```

6.1.13 slot

Construct an indexed slot from a representation.

The result is an ordinary dictionary, so its representation, index, and variance remain visible to Typst before an expression is constructed.

Parameters

```
slot(
  representation: dictionary,
  index: int str bytes,
  dual: bool none
) -> dictionary
```

6.1.13.0.1 representation `dictionary`

Representation dictionary.

6.1.13.0.2 index `int` or `str` or `bytes`

Abstract index label.

6.1.13.0.3 dual `bool` or `none`

Wrap the slot with Spenso's dual-index marker. `none` inherits the representation's `is-dual` field.

Default: `none`

6.1.14 metric

Construct the metric tensor for a representation.

Index labels are converted to slots automatically; existing slots pass through unchanged.

Parameters

```
metric(
  representation,
  first,
  second
) -> bytes
```

6.1.15 identity-tensor

Construct the identity tensor for a representation.

Parameters

```
identity-tensor(
  representation,
  first,
  second
) -> bytes
```

6.1.16 flat-tensor

Construct Spenso's musical isomorphism tensor for a representation.

Parameters

```
flat-tensor(
  representation,
  first,
  second
) -> bytes
```

6.1.17 dual-representation

Return the representation paired with this representation.

Parameters

```
dual-representation(representation) -> dictionary
```

6.2 Expression construction

6.2.1 add

Add symbolic expressions or constructor values.

Parameters

```
add(..terms) -> bytes
```

6.2.2 mul

Multiply symbolic expressions or constructor values.

Parameters

```
mul(..factors) -> bytes
```

6.2.3 neg

Negate a symbolic expression or constructor value.

Parameters

```
neg(expression) -> bytes
```

6.2.4 sub

Subtract two symbolic expressions or constructor values.

Parameters

```
sub(
  left,
  right
) -> bytes
```

6.2.5 div

Divide two symbolic expressions or constructor values.

Parameters

```
div(
  numerator,
  denominator
) -> bytes
```

6.2.6 pow

Raise a symbolic expression or constructor value to a power.

Parameters

```
pow(
  base,
  exponent
) -> bytes
```

6.3 Printing and inspection

6.3.1 print-settings

Create Spenso printer settings.

`"typst"` uses valid Typst math syntax with indexed tensor notation; `"compact"` uses Spenso's concise Symbolica-style form. Any field may be overridden without changing the others.

Parameters

```
print-settings(
  preset: str,
  with-dim: bool none,
  parens: bool none,
  commas: bool none,
  index-subscripts: bool none,
  symbol-scripts: bool none
) -> dictionary
```

6.3.1.0.1 preset `str`

Base preset: `"typst"` or `"compact"`.

Default: `"typst"`

6.3.1.0.2 with-dim `bool` or `none`

Include representation dimensions in printed slots.

Default: `none`

6.3.1.0.3 parens `bool` or `none`

Use parentheses around tensor arguments.

Default: `none`

6.3.1.0.4 commas `bool` or `none`

Separate tensor arguments with commas.

Default: `none`

6.3.1.0.5 index-subscripts `bool` or `none`

Render indices as subscripts where supported.

Default: `none`

6.3.1.0.6 symbol-scripts `bool` or `none`

Render non-slot tensor arguments as symbol scripts where supported.

Default: `none`

6.3.2 to-typst-source

Print an expression as Typst math source using Spenso's printer.

Parameters

```
to-typst-source(
  expression,
  settings: dictionary
) -> str
```

6.3.2.0.1 expression

Atom payload or constructor value.

6.3.2.0.2 settings `dictionary`

Settings created by `print-settings`.

Default: `_print-settings()`

6.3.3 to-typst

Print and evaluate an expression as Typst math content.

Parameters

```
to-typst(
  expression,
  settings: dictionary,
  block: bool
) -> content
```

6.3.3.0.1 expression

Atom payload or constructor value.

6.3.3.0.2 settings `dictionary`

Settings created by `print-settings`.

Default: `_print-settings()`

6.3.3.0.3 block `bool`

Display the result as a block equation.

Default: `false`

6.3.4 to-string

Print an expression in Spenso's compact Symbolica notation.

Parameters

```
to-string(
  expression,
  settings: dictionary
) -> str
```

6.3.4.0.1 expression

Atom payload or constructor value.

6.3.4.0.2 settings `dictionary`

Settings created by `print-settings`.

Default: `_print-settings(preset: "compact")`

6.3.5 inspect

Decode an Atom payload into a recursive CBOR expression tree.

The result uses `kind` values such as `symbol`, `number`, `function`, `sum`, `product`, and `power`. Function nodes include their name, arguments, and symmetry flags. This view is ordinary Typst data; transforming the expression still uses its lossless Atom payload.

Parameters

`inspect(expression) -> dictionary`

6.4 Metric and index transformations

6.4.1 cook-function

Cook a tensor function into Idenso's canonical structure.

Parameters

`cook-function(expression) -> bytes`

6.4.2 cook-indices

Cook indices into Idenso's canonical structure.

Parameters

`cook-indices(expression) -> bytes`

6.4.3 expand-bis

Selectively expand bispinor structures.

Parameters

`expand-bis(expression) -> bytes`

6.4.4 expand-metrics

Expand all supported metric structures.

Parameters

`expand-metrics(expression) -> bytes`

6.4.5 expand-mink

Selectively expand Minkowski structures.

Parameters

`expand-mink(expression) -> bytes`

6.4.6 expand-mink-bis

Selectively expand combined Minkowski and bispinor structures.

Parameters

`expand-mink-bis(expression) -> bytes`

6.4.7 list-dangling

Return the dangling indices as compatible Atom payloads.

Parameters

```
list-dangling(expression) -> array
```

6.4.8 simplify-metrics

Contract and simplify metric tensors.

Parameters

```
simplify-metrics(expression) -> bytes
```

6.4.9 to-dots

Rewrite supported contractions as dot products.

Parameters

```
to-dots(expression) -> bytes
```

6.4.10 wrap-dummies

Wrap dummy indices under the given header symbol.

Parameters

```
wrap-dummies(
  expression,
  header
) -> bytes
```

6.4.11 wrap-indices

Wrap all indices under the given header symbol.

Parameters

```
wrap-indices(
  expression,
  header
) -> bytes
```

6.5 Dirac and color transformations

6.5.1 dirac-adjoint

Take the Dirac adjoint of a tensor expression.

Parameters

```
dirac-adjoint(expression) -> bytes
```

6.5.2 expand-color

Selectively expand color structures.

Parameters

```
expand-color(expression) -> bytes
```

6.5.3 simplify-color

Simplify color tensors.

Parameters

```
simplify-color(expression) -> bytes
```

6.5.4 simplify-gamma

Simplify gamma-matrix expressions.

Parameters

```
simplify-gamma(expression) -> bytes
```

6.6 Advanced configuration

6.6.1 init

Create an independent Tydenso API.

The returned dictionary contains tensor constructors, Spenso-aware printers, CBOR inspection, and Idenso transformations. Use a custom `source` only when developing another compatible plugin; ordinary documents can use the imported top-level functions.

```
#let tensors = init()
#let V = (tensors.mink)(4)
#let p = (tensors.vector)("p")
#(tensors.to-typst)(p((tensors.slot)(V, "mu")))
```

p^{μ}

Parameters

```
init(source: str bytes none) -> dictionary
```

6.6.1.0.1 source `str` or `bytes` or `none`

WebAssembly plugin path or uncompressed module bytes. `none` selects the bundled compressed Tydenso engine.

Default: `none`

7 Compatibility and licensing

This manual describes Tydenso 0.1.0 and Typst 0.14 or newer. Tydenso's original source is released under the MIT License. The bundled engine also contains Spenso, Idenso, and Symbolica; their own upstream terms continue to apply. In particular, the MIT license for this interface does not relicense Symbolica. Read Symbolica's license before redistributing or deploying the WebAssembly bundle.

8 Acknowledgements

Tydenso would not exist without Symbolica, the exact algebra engine beneath its Atom representation. Thank you to Symbolica's contributors, and to the GammaLoop contributors for Spenso and Idenso, whose tensor model, printers, and identities define this package's mathematics.

Thanks also to Tidy, which powers the worked examples and generated reference in this manual.